



Facultad De Ingeniería

Escuela De Computación

Técnico en Ingeniería en Ciencias de la Computación

“Desafío Practico 02”

Diseño y Programación de Software Multiplataforma

Estudiantes – Grupo 3:

Gabriela Stephani Figueroa Calderón FC250109

Daniela Nicole García Guillén GG251303

Ciclo: 01 - 2026

Docente: Ing. Emerson Francisco Cartagena Candelario

Grupo: 01L

Fecha de realización: 10 / 04 / 2026

Índice

1.	Enlace de Repositorio	3
2.	Análisis del problema	4
2.1	Registrar piezas	4
2.2	Mostrar y ordenar la lista de piezas	4
2.3	Gestionar el borrado de registros	5
2.4	Mostrar los detalles de una pieza.....	5
2.5	Requisitos tecnológicos y de trabajo colaborativo	6
3.	Descripción del Código.....	7
3.1	App.js.....	7
3.2	usePieces.js	8
3.3	PieceForm.js.....	9
3.4	PieceList.js.....	13
3.5	PieceItem.js	14
3.6	DeleteButton.js.....	15
3.7	DetailModal.js.....	16
3.8	HomeScreen.js	18
4.	Resultado de Interfaces	22

1. Enlace de Repositorio

https://github.com/NicoleGG2008/Desafio_02_DPS.git

2. Análisis del problema

El sistema que se pretende desarrollar corresponde a una aplicación móvil para el registro y seguimiento de piezas de repuesto utilizadas en reparaciones. La aplicación debe permitir:

2.1 Registrar piezas

El sistema debe almacenar información de cada pieza cambiada, con los siguientes atributos:

- Tipo de pieza.
- Marca.
- Número de serie.
- Precio.
- Fecha de cambio.

Estos datos se almacenan en una estructura de tipo lista dentro del estado de la aplicación, sin requerir conexión a base de datos externa ni persistencia más allá de la sesión de uso.

2.2 Mostrar y ordenar la lista de piezas

En la pantalla principal de la aplicación se debe presentar una lista de las piezas registradas, ordenadas por fecha de cambio, de manera descendente (de la más reciente a la más antigua).

- Cada registro debe mostrar el tipo de pieza y la fecha de cambio.
- Si no existen piezas registradas, la aplicación debe mostrar el mensaje “No hay piezas, agregue una” en lugar de una lista vacía.

2.3 Gestionar el borrado de registros

El sistema debe incluir, por cada pieza de la lista, un botón de eliminación con la etiqueta “Eliminar”.

- Al tocar dicho botón, el registro correspondiente debe ser removido de la lista.
- Las piezas restantes deben mantenerse ordenadas por fecha de cambio, actualizando visualmente la interfaz de forma inmediata tras la eliminación

2.4 Mostrar los detalles de una pieza

Al seleccionar un registro de la lista (tocando la celda, excluyendo el botón de eliminación), se debe abrir un modal con la información completa de la pieza.

- Este modal debe mostrar todos los datos ingresados en el formulario original: tipo de pieza, marca, número de serie, precio y fecha de cambio.
- Debe incluir, como mínimo, un botón de “Cerrar” que permita regresar a la pantalla principal sin realizar cambios adicionales.
- Opcionalmente, se puede agregar un botón de “Eliminar” en el modal, permitiendo borrar la pieza desde esa vista detallada.

2.5 Requisitos tecnológicos y de trabajo colaborativo

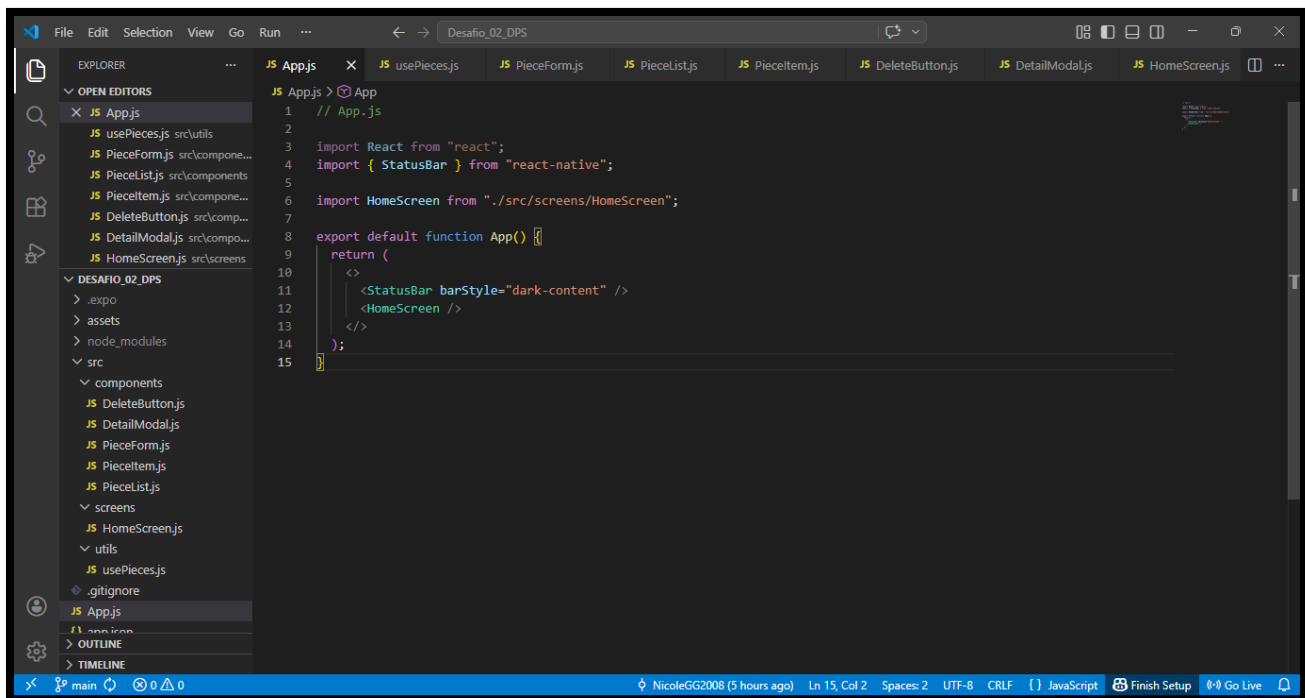
La aplicación debe desarrollarse exclusivamente utilizando React Native, orientada a entornos móviles, y no debe diseñarse ni depender de React para navegadores web.

- Debe ejecutarse en un dispositivo móvil (Android o iOS) o en un emulador compatible.
- El desarrollo debe realizarse en pareja, compartiendo un repositorio Git (por ejemplo, en GitHub o GitLab).
- El repositorio debe mostrar evidencia de contribuciones de ambos estudiantes, a través de commits asociados a sus respectivas cuentas.

3. Descripción del Código.

3.1 App.js

Es el punto de entrada de la aplicación.



The screenshot shows a code editor with the following content:

```
1 // App.js
2
3 import React from "react";
4 import { StatusBar } from "react-native";
5
6 import HomeScreen from "../src/screens/HomeScreen";
7
8 export default function App() {
9   return (
10     <>
11       <StatusBar barStyle="dark-content" />
12       <HomeScreen />
13     </>
14   );
15 }
```

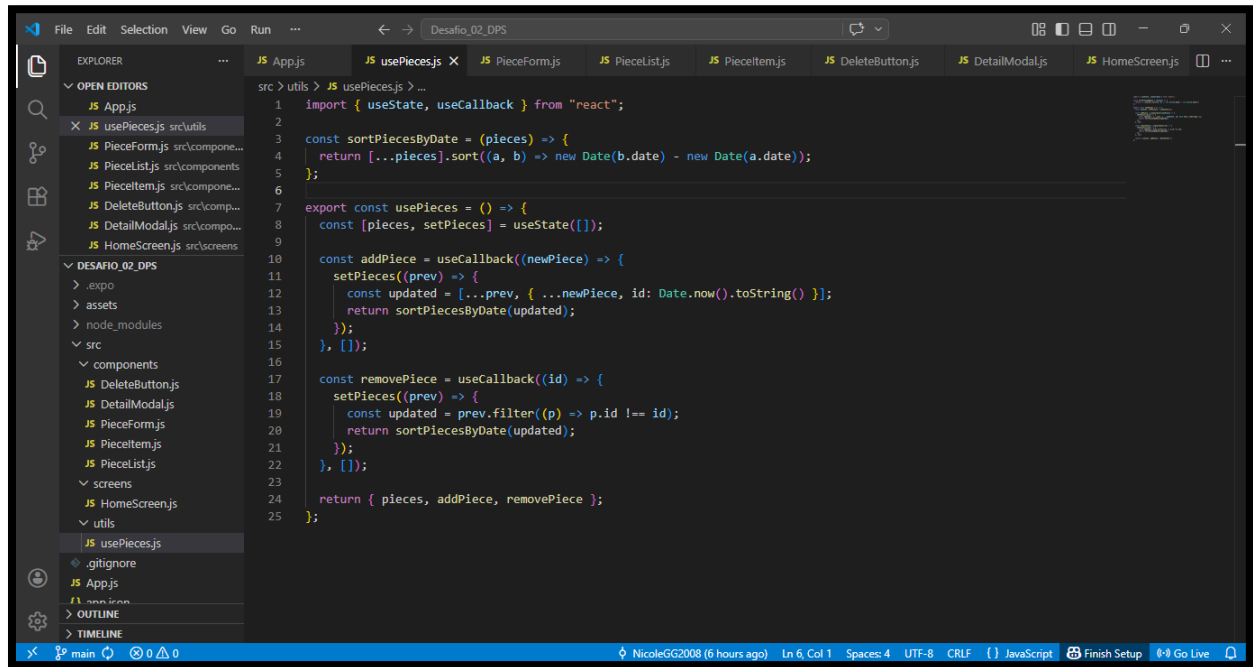
The Explorer panel on the left shows the project structure:

- Desafio_02_DPS
 - .expo
 - assets
 - node_modules
 - src
 - components
 - DeleteButton.js
 - DetailModal.js
 - PieceForm.js
 - PieceItem.js
 - PieceList.js
 - screens
 - HomeScreen.js
 - utils
 - usePieces.js
 - .gitignore
 - App.js

Muestra la pantalla principal (HomeScreen) y no contiene lógica de negocio directamente; solo se encarga de mostrar la interfaz principal y activar el sistema de componentes.

3.2 usePieces.js

Es el hook de estado de la aplicación.



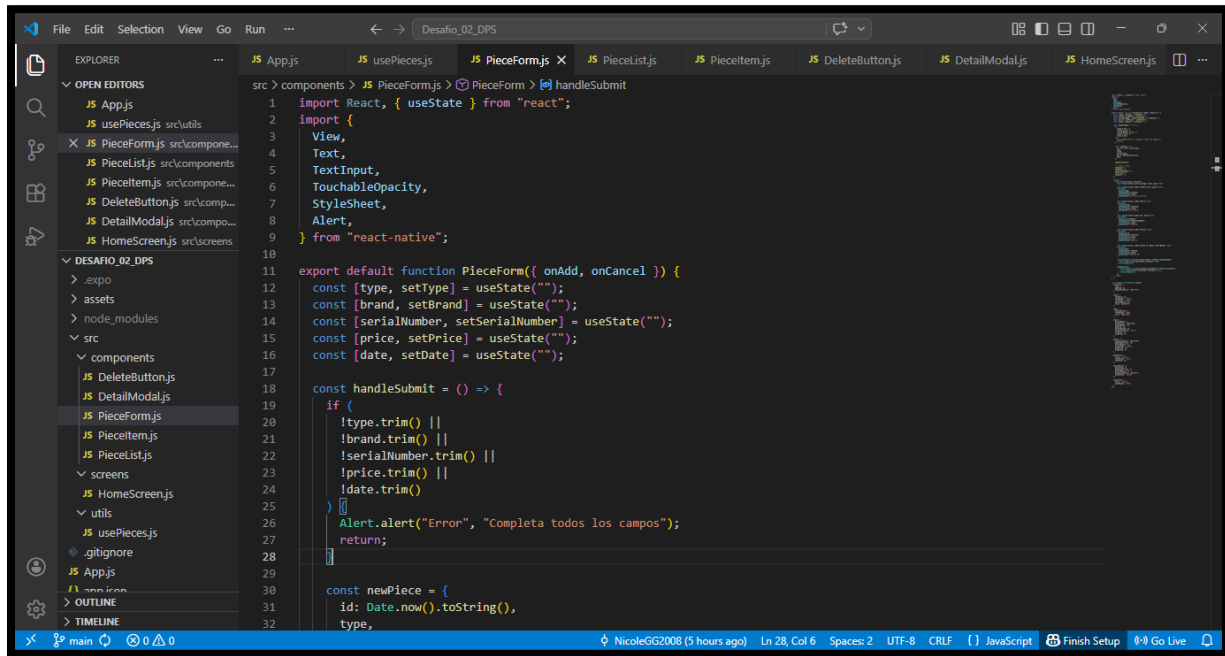
```
1 import { useState, useCallback } from "react";
2
3 const sortPiecesByDate = (pieces) => {
4   return [...pieces].sort((a, b) => new Date(b.date) - new Date(a.date));
5 };
6
7 export const usePieces = () => {
8   const [pieces, setPieces] = useState([]);
9
10  const addPiece = useCallback((newPiece) => {
11    setPieces((prev) => {
12      const updated = [...prev, { ...newPiece, id: Date.now().toString() }];
13      return sortPiecesByDate(updated);
14    });
15  }, []);
16
17  const removePiece = useCallback((id) => {
18    setPieces((prev) => {
19      const updated = prev.filter((p) => p.id !== id);
20      return sortPiecesByDate(updated);
21    });
22  }, []);
23
24  return { pieces, addPiece, removePiece };
25 };
```

Se encarga de:

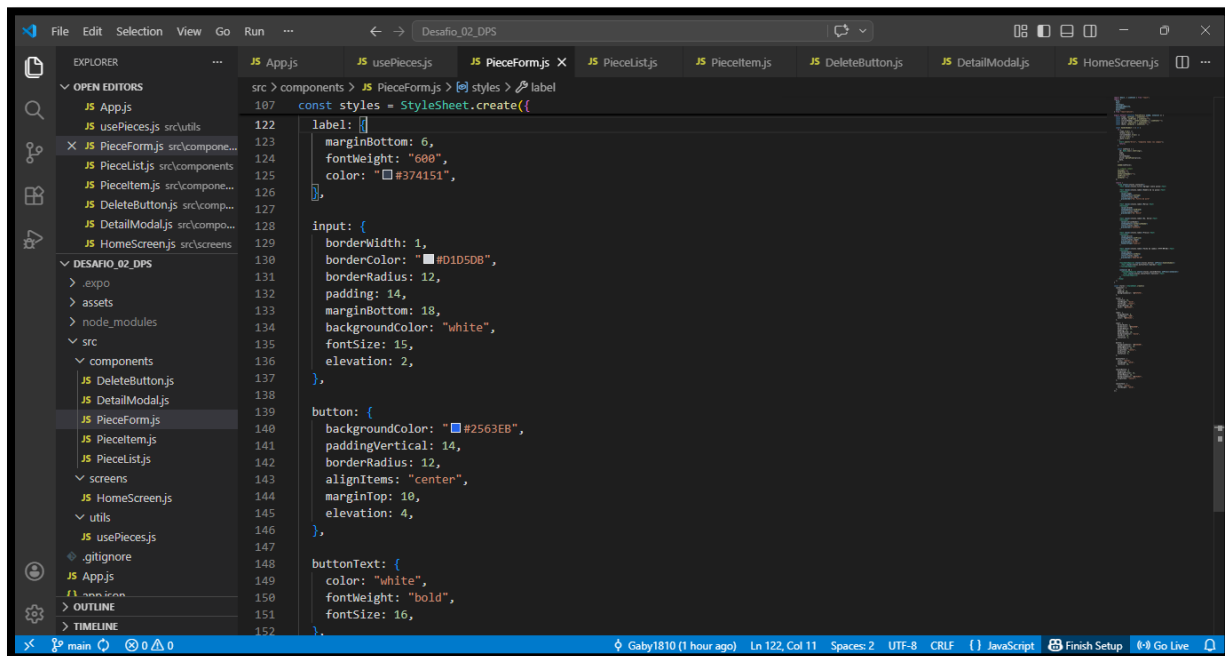
- Guardar la lista de piezas en el estado (pieces).
- Agregar una nueva pieza (addPiece) al formulario y mantenerla ordenada por fecha.
- Eliminar una pieza (removePiece) por su id y actualizar la lista ordenada.

3.3 PieceForm.js

Es el componente de formulario para registrar una pieza.



```
1 import React, { useState } from "react";
2 import {
3   View,
4   Text,
5   TextInput,
6   TouchableOpacity,
7   StyleSheet,
8   Alert,
9 } from "react-native";
10
11 export default function PieceForm({ onAdd, onCancel }) {
12   const [type, setType] = useState("");
13   const [brand, setBrand] = useState("");
14   const [serialNumber, setSerialNumber] = useState("");
15   const [price, setPrice] = useState("");
16   const [date, setDate] = useState("");
17
18   const handleSubmit = () => {
19     if (
20       !type.trim() ||
21       !brand.trim() ||
22       !serialNumber.trim() ||
23       !price.trim() ||
24       !date.trim()
25     ) {
26       Alert.alert("Error", "Completa todos los campos");
27       return;
28     }
29
30     const newPiece = {
31       id: Date.now().toString(),
32       type,
```



```
107 const styles = StyleSheet.create({
122   label: {
123     marginBottom: 6,
124     fontWeight: "bold",
125     color: "#374151",
126   },
127   input: {
128     borderWidth: 1,
129     borderColor: "#D1D5DB",
130     borderRadius: 12,
131     padding: 14,
132     marginBottom: 18,
133     backgroundColor: "white",
134     fontSize: 15,
135     elevation: 2,
136   },
137   button: {
138     backgroundColor: "#2563EB",
139     paddingVertical: 14,
140     borderRadius: 12,
141     alignItems: "center",
142     margin: 10,
143     elevation: 4,
144   },
145   buttonText: {
146     color: "white",
147     fontWeight: "bold",
148     fontSize: 16,
149   },
150 },
151 ),
152 ),
```

```
File Edit Selection View Go Run ...
Desafio_02_DPS

EXPLORER
src > components > JS PieceForm.js
JS App.js
JS usePieces.js src/utils
JS PieceForm.js src/compon...
JS PieceList.js src/components
JS PieceItem.js src/compon...
JS DeleteButton.js src/compo...
JS DetailModal.js src/compo...
JS HomeScreen.js src/screens
DESAFIO_02_DPS
  .expo
  assets
  node_modules
  src
    components
      JS DeleteButton.js
      JS DetailModal.js
      JS PieceForm.js
      JS PieceItem.js
      JS PieceList.js
    screens
      JS HomeScreen.js
    utils
      JS usePieces.js
  .gitignore
  App.js
  package.json
  OUTLINE
  TIMELINE

src > components > JS PieceForm.js > handleSubmit => newPiece
11 export default function PieceForm({ onAdd, onCancel }) {
18   const handleSubmit = () => {
30     const newPiece = {
33       brand,
34       serialNumber,
35       price: parseFloat(price),
36       date,
37     };
38     onAdd(newPiece);
39
40     // limpiar campos
41     setType("");
42     setBrand("");
43     setSerialNumber("");
44     setPrice("");
45     setDate("");
46   };
47
48   return (
49     <View style={styles.container}>
50       <Text style={styles.title}>Agregar nueva pieza</Text>
51       <Text style={styles.label}>Nombre de la pieza</Text>
52       <TextInput
53         value={type}
54         onChangeText={setType}
55         style={styles.input}
56         placeholder="Ej: Filtro de aire"
57       />
58       <Text style={styles.label}>Marca</Text>
59     </View>
60   );
61 }
```

```
File Edit Selection View Go Run ...
Desafio_02_DPS

EXPLORER
src > components > JS PieceForm.js
JS App.js
JS usePieces.js src/utils
JS PieceForm.js src/compon...
JS PieceList.js src/components
JS PieceItem.js src/compon...
JS DeleteButton.js src/compo...
JS DetailModal.js src/compo...
JS HomeScreen.js src/screens
DESAFIO_02_DPS
  .expo
  assets
  node_modules
  src
    components
      JS DeleteButton.js
      JS DetailModal.js
      JS PieceForm.js
      JS PieceItem.js
      JS PieceList.js
    screens
      JS HomeScreen.js
    utils
      JS usePieces.js
  .gitignore
  App.js
  package.json
  OUTLINE
  TIMELINE

src > components > JS PieceForm.js > PieceForm
11 export default function PieceForm({ onAdd, onCancel }) {
62   <TextInput
63     value={brand}
64     onChangeText={setBrand}
65     style={styles.input}
66     placeholder="Ej: Bosch"
67   />
68   <Text style={styles.label}>No. Serie</Text>
69   <TextInput
70     value={serialNumber}
71     onChangeText={setSerialNumber}
72     style={styles.input}
73     placeholder="12345XYZ"
74   />
75   <Text style={styles.label}>Precio</Text>
76   <TextInput
77     value={price}
78     onChangeText={setPrice}
79     style={styles.input}
80     placeholder="25.00"
81     keyboardType="numeric"
82   />
83   <Text style={styles.label}>Fecha de cambio (YYYY-MM-DD)</Text>
84   <TextInput
85     value={date}
86     onChangeText={setDate}
87     style={styles.input}
88     placeholder="2026-04-10"
89   />
90   </View>
91 }
92 }
```

```
src > components > JS PieceForm.js > PieceForm
11 export default function PieceForm({ onAdd, onCancel }) {
12   />
13   <TouchableOpacity style={styles.button} onPress={handleSubmit}>
14     <Text style={styles.buttonText}>Guardar</Text>
15   </TouchableOpacity>
16
17   {onCancel && (
18     <TouchableOpacity style={styles.cancelButton} onPress={onCancel}>
19       <Text style={styles.cancelText}>Cancelar</Text>
20     </TouchableOpacity>
21   )}
22 </View>
23 }
24
25 const styles = StyleSheet.create({
26   container: {
27     flex: 1,
28     padding: 25,
29     backgroundColor: "#F3F4F6",
30   },
31
32   title: {
33     fontSize: 22,
34     fontWeight: "bold",
35     textAlign: "center",
36     marginBottom: 25,
37     color: "#1F2937",
38   },
39
40   label: {
```

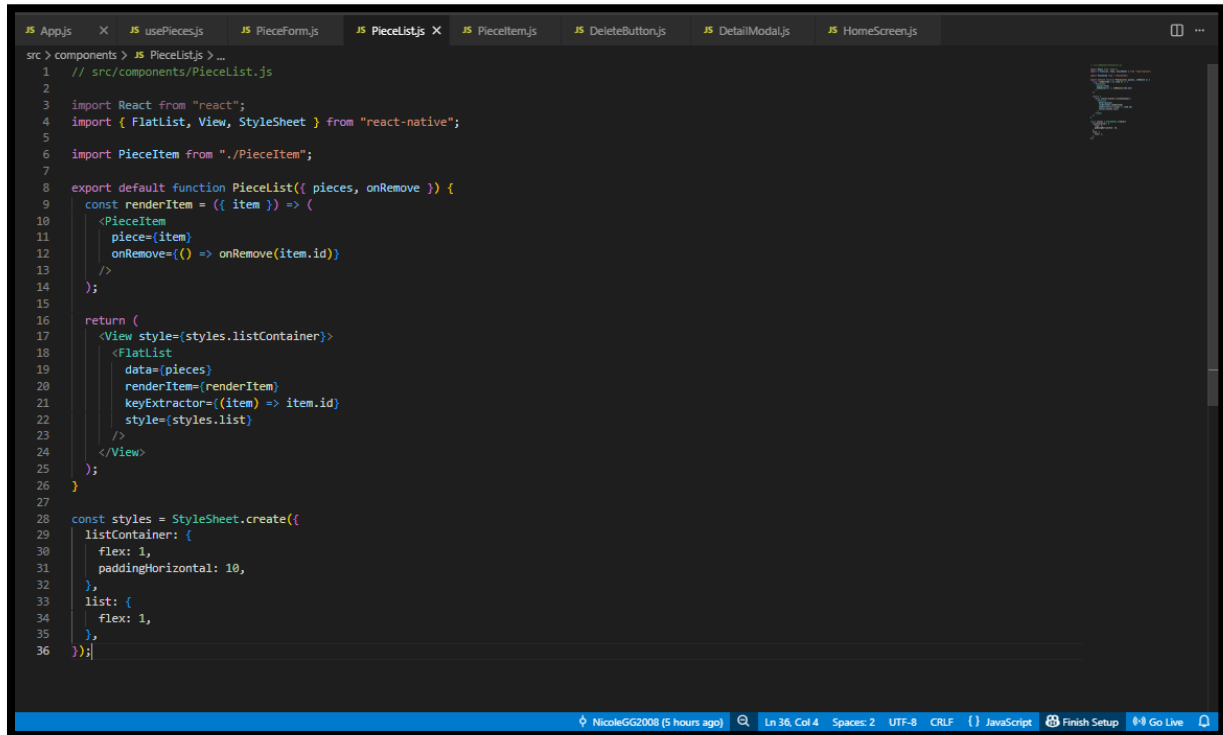
```
src > components > JS PieceForm.js > styles > label
107 const styles = StyleSheet.create({
108   buttonText: {
109     fontSize: 16,
110   },
111
112   cancelButton: {
113     marginTop: 12,
114     paddingVertical: 14,
115     borderRadius: 12,
116     backgroundColor: "#1F2937",
117     alignItems: "center",
118   },
119
120   cancelText: {
121     color: "white",
122     fontWeight: "bold",
123   },
124 });
```

Se encarga de:

- Mostrar campos de texto para:
 - Tipo de pieza
 - Marca
 - Número de serie
 - Precio
 - Fecha de cambio
- Validar que todos los campos estén completos antes de guardar.
- Enviar la nueva pieza al hook usePieces a través de onAdd.

3.4 PieceList.js

Es el componente que muestra la lista de piezas.



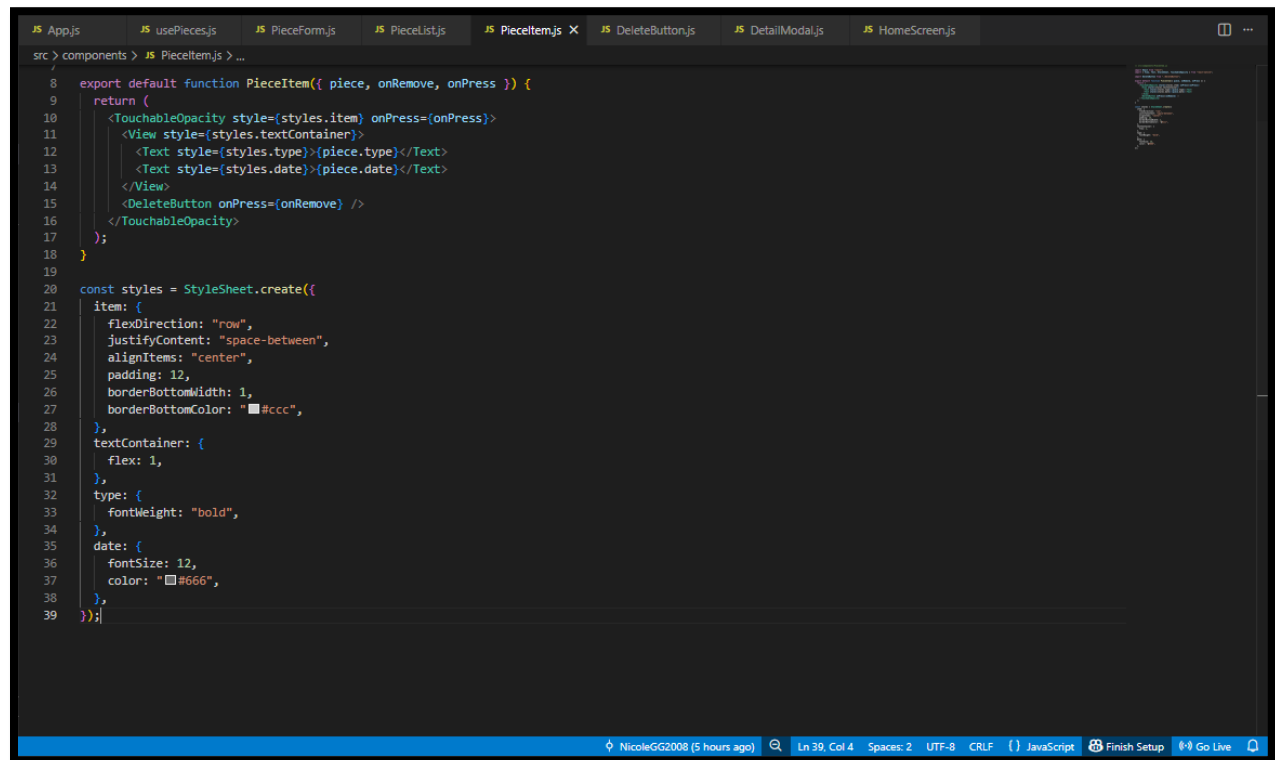
```
1 // src/components/PieceList.js
2
3 import React from "react";
4 import { FlatList, View, StyleSheet } from "react-native";
5
6 import PieceItem from "../PieceItem";
7
8 export default function PieceList({ pieces, onRemove }) {
9   const renderItem = ({ item }) => (
10     <PieceItem
11       piece={item}
12       onRemove={() => onRemove(item.id)}
13     />
14   );
15
16   return (
17     <View style={styles.listContainer}>
18       <FlatList
19         data={pieces}
20         renderItem={renderItem}
21         keyExtractor={(item) => item.id}
22         style={styles.list}
23       />
24     </View>
25   );
26 }
27
28 const styles = StyleSheet.create({
29   listContainer: {
30     flex: 1,
31     paddingHorizontal: 10,
32   },
33   list: {
34     flex: 1,
35   },
36 });
```

Se encarga de:

- Recibir la lista de piezas y la función de eliminación desde App.js.
- Usar FlatList para mostrar las piezas en orden por fecha.
- Usar el componente PieceItem para renderizar cada registro.

3.5 PieceItem.js

Es el componente que muestra un solo registro de pieza.



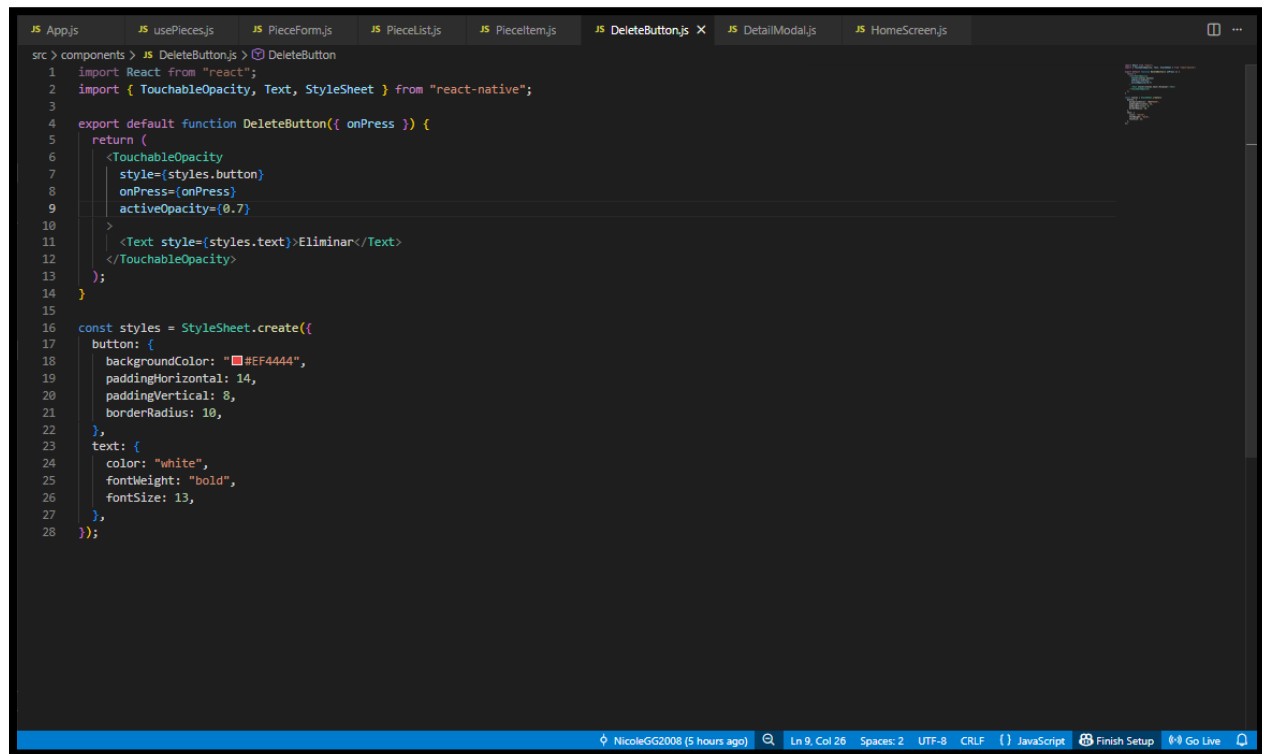
```
src > components > JS PieceItem.js > ...
8  export default function PieceItem({ piece, onRemove, onPress }) {
9    return (
10     <TouchableOpacity style={styles.item} onPress={onPress}>
11       <View style={styles.textContainer}>
12         <Text style={styles.type}>{piece.type}</Text>
13         <Text style={styles.date}>{piece.date}</Text>
14       </View>
15       <DeleteButton onPress={onRemove} />
16     </TouchableOpacity>
17   );
18 }
19
20 const styles = StyleSheet.create({
21   item: {
22     flexDirection: "row",
23     justifyContent: "space-between",
24     alignItems: "center",
25     padding: 12,
26     borderBottomWidth: 1,
27     borderBottomColor: "#ccc",
28   },
29   textContainer: {
30     flex: 1,
31   },
32   type: {
33     fontWeight: "bold",
34   },
35   date: {
36     fontSize: 12,
37     color: "#666",
38   },
39 });
```

Se encarga de:

- Mostrar el tipo de pieza y la fecha de cambio.
- Contener el botón DeleteButton que elimina el registro.
- Permite que al tocar el registro (sin tocar el botón) se abra el modal de detalles.

3.6 DeleteButton.js

Es un botón reutilizable para eliminar piezas.



```
src > components > JS DeleteButton.js > DeleteButton
1  import React from "react";
2  import { TouchableOpacity, Text, StyleSheet } from "react-native";
3
4  export default function DeleteButton({ onPress }) {
5    return (
6      <TouchableOpacity
7        style={styles.button}
8        onPress={onPress}
9        activeOpacity={0.7}
10      >
11        <Text style={styles.text}>Eliminar</Text>
12      </TouchableOpacity>
13    );
14  }
15
16  const styles = StyleSheet.create({
17    button: {
18      backgroundColor: "#EF4444",
19      paddingHorizontal: 14,
20      paddingVertical: 8,
21      borderRadius: 10,
22    },
23    text: {
24      color: "white",
25      fontWeight: "bold",
26      fontSize: 13,
27    },
28  });
```

Se encarga de:

- Mostrar el texto “Eliminar”.
- Recibir una función onPress (que viene del hook usePieces) y ejecutarla cuando se toca el botón.

3.7 DetailModal.js

Es el componente que muestra los detalles de una pieza en un modal.

```
JS App.js JS usePieces.js JS PieceForm.js JS PieceList.js JS PieceItem.js JS DeleteButton.js JS DetailModal.js X JS HomeScreen.js
src > components > JS DetailModal.js > ...
1 import React from "react";
2 import {
3   View,
4   Text,
5   Modal,
6   TouchableOpacity,
7   StyleSheet,
8   ScrollView,
9 } from "react-native";
10
11 export default function DetailModal({ isVisible, onClose, piece, onDelete }) {
12   if (!piece) return null;
13
14   return (
15     <Modal
16       visible={isVisible}
17       transparent={true}
18       animationType="slide"
19       onRequestClose={onClose}
20     >
21       <View style={styles.modalContainer}>
22         <View style={styles.modalContent}>
23           <Text style={styles.title}>Detalle de la pieza</Text>
24           <Text style={styles.subtitle}>Información de la pieza</Text>
25           <ScrollView style={styles.scroll}>
26             <Text style={styles.detailRow}>
27               <Text style={styles.label}>Pieza: </Text> {piece.type}
28             </Text>
29             <Text style={styles.detailRow}>
30               <Text style={styles.label}>Marca: </Text> {piece.brand}
31             </Text>
32             <Text style={styles.detailRow}>
33               <Text style={styles.label}>No. Serie: </Text> {piece.serialNumber}
34             </Text>
35             <Text style={styles.detailRow}>
36               <Text style={styles.label}>Precio: </Text> ${piece.price}
37             </Text>
38             <Text style={styles.detailRow}>
39               <Text style={styles.label}>Fecha de cambio: </Text> {piece.date}
```

```
JS App.js JS usePieces.js JS PieceForm.js JS PieceList.js JS PieceItem.js JS DeleteButton.js JS DetailModal.js X JS HomeScreen.js
src > components > JS DetailModal.js > DetailModal
11 export default function DetailModal({ isVisible, onClose, piece, onDelete }) {
40   </Text>
41   </ScrollView>
42   <View style={styles.bottomButtons}>
43     <TouchableOpacity
44       style={styles.deleteBtn}
45       onPress={() => {
46         onDelete(piece.id);
47         onClose();
48       }}
49     >
50       <Text style={styles.btnText}>Eliminar</Text>
51     </TouchableOpacity>
52     <TouchableOpacity style={styles.closeBtn} onPress={onClose}>
53       <Text style={styles.btnText}>Cerrar</Text>
54     </TouchableOpacity>
55   </View>
56 </View>
57 </View>
58 </Modal>
59 </>
60 }
61
62 const styles = StyleSheet.create({
63   modalContainer: {
64     flex: 1,
65     justifyContent: "center",
66     alignItems: "center",
67     backgroundColor: "rgba(0,0,0,0.5)",
68   },
69
70   modalContent: {
71     width: "90%",
72     maxHeight: "75%",
73     backgroundColor: "white",
74     borderRadius: 20,
75     padding: 25,
76     elevation: 10,
77   },
78 });
```

```
src > components > JS DetailModal.js > styles
62  const styles = StyleSheet.create({
78
79    title: {
80      fontSize: 22,
81      fontWeight: "bold",
82      textAlign: "center",
83      marginBottom: 5,
84      color: "#1F2937",
85    },
86
87    subtitle: {
88      fontSize: 14,
89      color: "#687280",
90      textAlign: "center",
91      marginBottom: 20,
92    },
93
94    scroll: {
95      marginBottom: 10,
96    },
97
98    detailRow: {
99      fontSize: 16,
100     marginBottom: 10,
101     lineHeight: 24,
102     color: "#374151",
103   },
104
105   label: {
106     fontWeight: "bold",
107     color: "#1F2937",
108   },
109
110   bottomButtons: {
111     flexDirection: "row",
112     justifyContent: "space-between",
113     marginTop: 15,
114   },
115 }
```

```
122
123   closeBtn: {
124     backgroundColor: "#2563EB",
125     paddingHorizontal: 25,
126     paddingVertical: 12,
127     borderRadius: 12,
128   },
129
130   btnText: {
131     color: "white",
132     fontWeight: "bold",
133   },
134 }
```

Se encarga de:

- Recibir los datos de la pieza seleccionada.
- Mostrar todos los campos ingresados en el formulario dentro de un modal tipo “factura”.
- Incluir un botón de “Cerrar” y, opcionalmente, “Eliminar” para gestionar el registro.

3.8 HomeScreen.js

Es la pantalla principal de la aplicación.

```
JS App.js JS usePieces.js JS PieceForm.js JS PieceList.js JS PieceItem.js JS DeleteButton.js JS DetailModal.js JS HomeScreen.js X
src > screens > JS HomeScreen.js > HomeScreen
1 import React, { useState } from "react";
2 import {
3   View,
4   Text,
5   FlatList,
6   TouchableOpacity,
7   Modal,
8   StyleSheet,
9 } from "react-native";
10
11 import { usePieces } from "../utils/usePieces";
12 import PieceForm from "../components/PieceForm";
13 import DetailModal from "../components/DetailModal";
14
15 export default function HomeScreen() {
16   const { pieces, addPiece, removePiece } = usePieces();
17   const [showForm, setShowForm] = useState(false);
18   const [selectedPiece, setSelectedPiece] = useState(null);
19
20   const handleAddPiece = (newPiece) => {
21     addPiece(newPiece);
22     setShowForm(false);
23   };
24
25   const handleDelete = (id) => {
26     removePiece(id);
27     setSelectedPiece(null);
28   };
29
30   return (
31     <View style={styles.container}>
32       /* HEADER */
33       <View style={styles.header}>
34         <Text style={styles.title}>Piezas</Text>
35
36         <TouchableOpacity
37           style={styles.addButton}
38           onPress={() => setShowForm(true)}
39         />

```

```
JS App.js JS usePieces.js JS PieceForm.js JS PieceList.js JS PieceItem.js JS DeleteButton.js JS DetailModal.js JS HomeScreen.js X
src > screens > JS HomeScreen.js > HomeScreen
15 export default function HomeScreen() {
40   >
41     <Text style={styles.addButtonText}>+ Agregar pieza</Text>
42   </TouchableOpacity>
43 </View>
44
45   /* HISTORIAL */
46   <View style={styles.historyContainer}>
47     <Text style={styles.historyTitle}>Historial de piezas</Text>
48
49     {pieces.length === 0 ? (
50       <Text style={styles.emptyText}>
51         No hay piezas, agregue una
52       </Text>
53     ) : (
54       <FlatList
55         data={pieces}
56         keyExtractor={(item) => item.id}
57         renderItem={({ item }) => (
58           <TouchableOpacity
59             style={styles.pieceItem}
60             onPress={() => setSelectedPiece(item)}
61           >
62             <View style={styles.pieceMain}>
63               <Text style={styles.pieceType}>{item.type}</Text>
64               <Text style={styles.pieceDate}>{item.date}</Text>
65             </View>
66
67             <TouchableOpacity
68               style={styles.deleteButton}
69               onPress={() => handleDelete(item.id)}
70             >
71               <Text style={styles.deleteText}>Eliminar</Text>
72             </TouchableOpacity>
73           </TouchableOpacity>
74         )}
75       </FlatList>
76     )}
77 </View>

```

```
JS App.js JS usePieces.js JS PieceForm.js JS PieceList.js JS PieceItem.js JS DeleteButton.js JS DetailModal.js JS HomeScreen.js X
src > screens > JS HomeScreen.js > HomeScreen
15 export default function HomeScreen() {
78
79   /* MODAL FORMULARIO */
80   <Modal visible={showForm} animationType="slide">
81     <View style={styles.modalContainer}>
82       <PieceForm
83         onAdd={handleAddPiece}
84         onCancel={() => setShowForm(false)}
85       />
86     </View>
87   </Modal>
88
89   /* MODAL DETALLE */
90   <DetailModal
91     isVisible={!selectedPiece}
92     onClose={() => setSelectedPiece(null)}
93     piece={selectedPiece}
94     onDelete={handleDelete}
95   />
96 </View>
97 };
98
99
100 const styles = StyleSheet.create({
101   container: {
102     flex: 1,
103     backgroundColor: "#F3F4F6",
104   },
105
106   header: {
107     paddingTop: 60,
108     paddingBottom: 30,
109     paddingHorizontal: 20,
110     backgroundColor: "#1F2937",
111     borderBottomLeftRadius: 30,
112     borderBottomRightRadius: 30,
113     elevation: 8,
114   },
115
116 });
```

```
JS App.js JS usePieces.js JS PieceForm.js JS PieceList.js JS PieceItem.js JS DeleteButton.js JS DetailModal.js JS HomeScreen.js X
src > screens > JS HomeScreen.js > styles
100 const styles = StyleSheet.create({
116   title: {
117     fontSize: 28,
118     fontWeight: "bold",
119     color: "white",
120   },
121
122   addButton: {
123     marginTop: 15,
124     backgroundColor: "#22C55E",
125     paddingVertical: 12,
126     borderRadius: 12,
127     alignItems: "center",
128   },
129
130   addButtonText: {
131     color: "white",
132     fontWeight: "bold",
133     fontSize: 16,
134   },
135
136   historyContainer: {
137     flex: 1,
138     padding: 20,
139   },
140
141   historyTitle: {
142     fontSize: 18,
143     fontWeight: "600",
144     marginBottom: 15,
145     color: "#1F2937",
146   },
147
148   emptyText: {
149     textAlign: "center",
150     marginTop: 40,
151     color: "#6B7280",
152     fontSize: 16,
153   },
154
155 });
```

```
JS App.js JS usePieces.js JS PieceForm.js JS PieceList.js JS PieceItem.js JS DeleteButton.js JS DetailModal.js JS HomeScreen.js X
src > screens > JS HomeScreen.js > styles
100 const styles = StyleSheet.create({
154
155   pieceItem: {
156     flexDirection: "row",
157     justifyContent: "space-between",
158     alignItems: "center",
159     padding: 15,
160     backgroundColor: "white",
161     borderRadius: 15,
162     marginBottom: 12,
163     elevation: 4,
164   },
165
166   pieceMain: {
167     flex: 1,
168   },
169
170   pieceType: {
171     fontSize: 17,
172     fontWeight: "bold",
173     color: "#111827",
174   },
175
176   pieceDate: {
177     fontSize: 13,
178     color: "#6B7280",
179     marginTop: 4,
180   },
181
182   deleteButton: {
183     backgroundColor: "#EF4444",
184     paddingHorizontal: 14,
185     paddingVertical: 8,
186     borderRadius: 10,
187   },
188
189   deleteText: {
190     color: "white",
191     fontWeight: "bold",
192   },
193
194   modalContainer: {
195     flex: 1,
196     backgroundColor: "#F3F4F6",
197   },
198 });
```

```
JS App.js JS usePieces.js JS PieceForm.js JS PieceList.js JS PieceItem.js JS DeleteButton.js JS DetailModal.js JS HomeScreen.js X
src > screens > JS HomeScreen.js > styles > deleteText
100 const styles = StyleSheet.create({
189   deleteText: {
190     color: "white",
191     fontWeight: "bold",
192   },
193
194   modalContainer: {
195     flex: 1,
196     backgroundColor: "#F3F4F6",
197   },
198 });
```

Se encarga de:

- Mostrar el título “Piezas” y el botón “Agregar piezas”.
- Mostrar el historial de piezas con el mensaje “No hay piezas, agregue una” si no hay registros.
- Controlar el uso de modales:
 - El formulario de agregar pieza (PieceForm).
 - El modal de detalles de la pieza (DetailModal).

4. Resultado de Interfaces



Agregar nueva pieza

Nombre de la pieza

Ej: Filtro de aire

Marca

Ej: Bosch

No. Serie

12345XYZ

Precio

25.00

Fecha de cambio (YYYY-MM-DD)

2026-04-10

Guardar

Cancelar

Agregar nueva pieza

Nombre de la pieza

Filtro de Aire

Marca

Bosh

No. Serie

123XYZ

Precio

15

Fecha de cambio (YYYY-MM-DD)

2026-04-17

Guardar

Cancelar

Agregar nueva pieza

Nombre de la pieza

Filtro de Aire

Marca

Bosh

No. Serie

123XYZ

Precio

15

Fecha de cambio (YYYY-MM-DD)

2026-04-17

Guardar

Cancelar

Piezas

+ Agregar pieza

Historial de piezas

Filtro de Aire

2026-04-17

Eliminar

Piezas

+ Agregar pieza

Historial de piezas

Filtro de Aire

Detalle de la pieza

Información de la pieza

Pieza: Filtro de Aire

Marca: Bosh

No. Serie: 123XYZ

Precio: \$15

Fecha de cambio: 2026-04-17

Eliminar

Cerrar